

Pourquoi lire ça avant la séance ?

En séance on ne fait QUE pratiquer - pas de cours. Si tu lis ce document avant, tu arrives avec les bases en tête et tu profites vraiment des exercices. Compte environ 20 minutes de lecture.

1	La structure d'un Pod
2	Labels et namespaces
3	restartPolicy
4	Init containers
5	Variables d'environnement
6	Ressources (CPU & mémoire)
7	Commandes kubectl essentielles
8	Le réflexe dry-run
9	Checklist avant la séance

1 La structure d'un Pod

Un Pod, c'est la plus petite chose que tu peux créer dans Kubernetes. Pense-y comme une enveloppe qui contient un ou plusieurs containers. La plupart des manifests que tu écriras à l'examen suivent cette structure : `apiVersion`, `kind`, `metadata`, `spec`.

```
apiVersion: v1          # version de l'API - toujours v1 pour les Pods
kind: Pod               # le type de ressource
metadata:
  name: mon-pod         # le nom de ton Pod
  namespace: mon-ns     # optionnel : dans quel espace il vit
  labels:               # optionnel : des étiquettes pour identifier le Pod
    app: mon-app
spec:
  containers:
    - name: mon-container # le nom du container (différent du Pod !)
      image: nginx:1.25   # l'image Docker à utiliser
```

C'est un Pod simple et complet. Le strict minimum serait : `apiVersion`, `kind`, `metadata.name`, puis `spec.containers` avec au moins `name` et `image`. Le reste (variables d'env, ressources, init containers...) vient s'ajouter dans `spec`.

piège `apiVersion` pour un Pod = `v1`. Pas `apps/v1` - c'est pour les Deployments.

piège `containers` est une liste YAML (avec un tiret -), pas juste un bloc.

2 Labels et namespaces

Les labels sont des étiquettes que tu colles sur tes ressources. Ça sert à les identifier, les filtrer, et plus tard à les cibler (un Service retrouve ses Pods grâce aux labels). Un label, c'est juste une paire clé/valeur, tu peux en mettre autant que tu veux.

```
metadata:
  labels:
    app: frontend      # identifie l'application
    env: prod          # identifie l'environnement
    version: v1.2      # tu peux en ajouter autant que tu veux
```

Les namespaces, c'est comme des dossiers dans le cluster - ils isolent les ressources entre équipes ou environnements. Si tu ne précises pas de namespace, Kubernetes met ton Pod dans `default`.

```
# Créer un namespace
kubectl create ns mon-namespace

# Voir tous les namespaces
kubectl get ns

# Toujours préciser -n quand tu travailles avec un namespace
kubectl get pod mon-pod -n mon-namespace
kubectl get pod mon-pod -n monitoring --show-labels
```

piège

Oublier -n <namespace> : kubectl cherche dans default et répond 'not found'. C'est le piège le plus fréquent à l'exam.

3

restartPolicy - que fait Kubernetes quand ton container s'arrête ?

Ce champ contrôle ce que Kubernetes fait si ton container plante ou se termine. Il y a 3 valeurs possibles :

Valeur	Ce que ça fait	Quand l'utiliser
Always	Redémarre toujours, succès ou échec	Serveurs, applis longue durée - c'est la valeur par défaut
OnFailure	Redémarre seulement si le container a planté (exit != 0)	Tâches qui peuvent échouer et réessayer ; le cas Job sera vu plus tard
Never	Ne redémarre jamais, quoi qu'il arrive	Tâche one-shot, script qu'on lance une seule fois

À retenir

Si l'énoncé dit 'le Pod doit tourner en permanence' -> Always. 'Doit s'arrêter après' -> Never. 'Peut réessayer si ça échoue' -> OnFailure.

4

Init containers - faire quelque chose avant de démarrer

Parfois ton app a besoin qu'une tâche soit faite avant de démarrer - attendre une base de données, préparer des fichiers, vérifier une config. C'est le rôle des init containers : ils s'exécutent avant les containers principaux, dans l'ordre, et le Pod attend qu'ils terminent avec succès.

```
spec:
  initContainers:
    - name: init-attente
      image: busybox
      command: ["echo", "init terminé"]
  containers:
    - name: mon-app
      image: nginx:1.25
      # mon-app ne démarre QUE si init-attente a réussi
```

Pour voir ce qui se passe pendant l'init :

```
kubectl get pod mon-pod -n mon-ns
# Pendant l'init -> Init:0/1
# Après l'init -> Running
```

piège

initContainers (avec un 's') se place au même niveau que containers dans spec - pas à l'intérieur de containers.

5 Variables d'environnement - passer des configs à ton app

Les variables d'env, c'est le moyen standard de passer des configurations à un container (URL de base de données, mode prod/dev, clés d'API...). En YAML elles se déclarent sous env, qui est une liste de paires name/valeur.

```
containers:
- name: mon-app
  image: busybox
  env:
    - name: APP_ENV      # nom de la variable
      value: "production" # sa valeur
    - name: LOG_LEVEL
      value: "info"
```

Pour vérifier que tes variables sont bien passées après le deploy :

```
kubectl describe pod mon-pod -n mon-ns
# Cherche la section 'Environment' dans la sortie

kubectl exec mon-pod -n mon-ns -- env
# Liste toutes les variables d'env dans le container
```

piège

env n'est pas un dictionnaire clé: valeur - c'est une liste de blocs {name, value}. La syntaxe avec un tiret - devant chaque 'name' est obligatoire.

6 Ressources CPU et mémoire - donner des limites à ton Pod

Kubernetes a besoin de savoir combien de ressources ton Pod consomme pour pouvoir le placer sur le bon node. Il y a deux notions importantes :

Champ	C'est quoi ?
requests	Ce que ton Pod demande au scheduler pour trouver un node. Kubernetes garantit que tu auras au minimum ça.
limits	Le maximum que ton container peut consommer. Au-delà, il est tué (OOMKilled pour la mémoire).

```
containers:
- name: mon-app
  image: busybox
  resources:
    requests:
      memory: "64Mi" # minimum garanti
      cpu: "125m"     # 0.125 vCPU
    limits:
      memory: "128Mi" # maximum autorisé
      cpu: "250m"     # 0.25 vCPU
```

Notation	Équivalent	Exemple concret
1	1 vCPU complet	Un coeur entier
500m	0.5 vCPU	Un demi-coeur
250m	0.25 vCPU	Un quart de coeur

piège

cpu: 250 sans le 'm' = 250 vCPU. Ton Pod ne sera jamais schedulé. Toujours mettre le 'm' pour les millicores.

piège

limits sans requests -> Kubernetes copie automatiquement les limits dans requests, ce qui peut bloquer le scheduling si les valeurs sont trop élevées.

7

Commandes kubectl - ton couteau suisse

Créer et appliquer

```
kubectl apply -f pod.yaml # crée ou met à jour ; à privilégier pour un YAML modifié
kubectl create -f pod.yaml # crée seulement ; échoue si la ressource existe déjà
```

**tip
exam**

À l'exam, privilégie apply pour appliquer un YAML que tu as modifié. Si tu resoumets par erreur, ça ne plante pas.

Inspecter un Pod

```
kubectl get pod mon-pod -n mon-ns
kubectl get pod mon-pod -n mon-ns -o wide      # voir aussi le node
kubectl get pod mon-pod -n mon-ns --show-labels # voir les labels
kubectl get pod mon-pod -n mon-ns -o yaml      # voir tout le manifest
kubectl describe pod mon-pod -n mon-ns        # détails complets + Events
```

Lire les logs

```
kubectl logs mon-pod -n mon-ns
kubectl logs mon-pod -n mon-ns --previous      # logs avant le dernier crash
kubectl logs mon-pod -n mon-ns -c mon-container # si plusieurs containers
```

Entrer dans un container

```
kubectl exec -it mon-pod -n mon-ns -- sh
kubectl exec mon-pod -n mon-ns -- env          # lister les variables d'env
```

La doc intégrée - kubectl explain

À l'exam tu n'as pas accès à Google. kubectl explain est ta documentation :

```
kubectl explain pod.spec
kubectl explain pod.spec.containers
kubectl explain pod.spec.containers.resources
kubectl explain pod.spec.initContainers
```

astuce

Tape `kubectl explain pod.spec` et lis la liste des champs disponibles. C'est comme avoir la doc officielle intégrée dans le terminal.

Pod de debug éphémère

Besoin de tester quelque chose dans le cluster sans laisser de trace ?

```
kubectl run debug --image=busybox \
  --restart=Never --rm -it \
  -n mon-ns -- sh
# --restart=Never : ne redémarre pas
# --rm : supprimé automatiquement quand tu quittes
```

Extraire un champ précis

Certaines questions demandent d'écrire la valeur d'un champ dans un fichier. jsonpath évite de lire tout le YAML :

```
kubectl get pod mon-pod -n mon-ns \
  -o jsonpath='{.status.phase}'

kubectl get pod mon-pod -n mon-ns \
  -o jsonpath='{.spec.containers[0].image}'
```

Changer de cluster - le réflexe context switch

À l'exam tu travailles sur plusieurs clusters. Avant chaque question, tu dois switcher de contexte :

```
kubectl config get-contexts      # liste tous les contextes
kubectl config use-context k8s-prod # switcher vers k8s-prod
kubectl config current-context    # vérifier où tu es
```

**tip
exam**

Toujours faire use-context avant de commencer une question. Créer un Pod dans le mauvais cluster = réponse fausse même si le YAML est parfait.

8 Le réflexe dry-run - ne jamais écrire de YAML from scratch

Écrire un Pod YAML de zéro à l'exam, c'est lent et risqué. La bonne méthode, c'est de le générer avec kubectl, puis de le modifier seulement si nécessaire.

```
# Générer le YAML sans rien créer dans le cluster
kubectl run mon-pod --image=nginx:1.25 \
  -n mon-ns \
  --labels="app=frontend,env=prod" \
  --dry-run=client -o yaml > pod.yaml

# Modifier pod.yaml si besoin, puis appliquer
kubectl apply -f pod.yaml
kubectl get pod mon-pod -n mon-ns
```

Aliases à configurer dès la première minute de l'exam

Ces raccourcis te font gagner plusieurs minutes sur l'ensemble des questions :

```
alias k=kubectl
export do="--dry-run=client -o yaml"
export now="--force --grace-period=0" # à utiliser avec prudence

# Utilisation
k run mon-pod --image=nginx $do > pod.yaml
k delete pod mon-pod $now
```

piège

--dry-run sans =client est déprécié -> toujours --dry-run=client.

piège

Oublier -o yaml > fichier.yaml -> rien n'est écrit sur disque.

piège

dry-run ne crée rien dans le cluster. Toujours enchaîner avec kubectl apply.

9 Checklist - tu es prêt si tu coches tout ça

Avant d'arriver en séance, vérifie que tu es à l'aise sur chaque point :

1	Je connais la structure habituelle d'un manifest CKAD (apiVersion, kind, metadata, spec)
2	Je sais que apiVersion d'un Pod = v1, pas apps/v1
3	Je comprends la différence restartPolicy: Never / Always / OnFailure
4	Je sais ce qu'est un init container et pourquoi on l'utilise
5	Je sais écrire des variables d'env sous forme de liste name/value
6	Je connais la différence entre requests et limits
7	Je sais lire les logs d'un container crashé avec --previous
8	Je sais utiliser kubectl describe et lire la section Events
9	Je connais la commande dry-run=client -o yaml
10	Je sais switcher de contexte avec kubectl config use-context

Scope séance 1 : Pods uniquement · Pas de Services, Volumes ni Probes - ce sera les séances suivantes.